

Day 17 - Path Traversal Lab

Internship: Trios Cyber

Date: 17 September 2026

Task: Path Traversal Lab

Platform: PortSwigger Web Security Academy

Tool: Burp Suite

Status: Completed

1. Objective

The objective of Day 17 was to complete two authorized PortSwigger Web Security Academy path traversal laboratories using Burp Suite.

The exercises focused on identifying a vulnerable file-related parameter and demonstrating how insufficient path validation can allow access to files outside the intended application directory.

2. Authorized Scope

All testing performed during this task was restricted to the designated PortSwigger Web Security Academy training laboratories.

No production systems, third-party applications, or unauthorized hosts were tested.

3. Tools and Technologies

- PortSwigger Web Security Academy
 - Burp Suite
 - Burp Proxy
 - Burp Repeater
 - Firefox browser
 - HTTP requests and responses
-

4. Lab 1 - File Path Traversal, Simple Case

4.1 Lab Objective

The first laboratory demonstrated a basic path traversal vulnerability in an image-loading functionality.

The objective was to identify the parameter responsible for selecting the requested image and determine whether it could be manipulated to access a file outside the intended directory.

4.2 Vulnerable Parameter

During HTTP request inspection in Burp Suite, image requests were observed using the following structure:

```
GET /image?filename=<image-file>
```

The identified vulnerable parameter was:

filename

The relevant endpoint was:

/image ## 4.3 Testing Method

A normal image request was examined in Burp Repeater. The value of the filename parameter was then modified to use relative path traversal sequences.

The tested traversal payload was:

```
../../../../etc/passwd ## 4.4 Result
```

The modified request returned the contents of the Linux /etc/passwd file.

The response included the root account entry:

```
root:x:0:0:root:/root:/bin/bash
```

This demonstrated that the application accepted a user-controlled relative filesystem path and resolved it outside the intended image directory.

4.5 Evidence

01-lab-1-initial-page.png - Initial laboratory page 02-lab-1-path-traversal-success.png - Successful path traversal response # 5.
Lab 2 - Traversal Sequences Blocked with Absolute Path Bypass
5.1 Lab Objective

The second laboratory demonstrated a different path traversal validation weakness.

The exercise focused on a situation where relative traversal sequences are restricted, while an absolute filesystem path may still be accepted.

5.2 Vulnerable Parameter

Inspection of the application source and image references identified requests using:

```
/image?filename=5.jpg
```

The vulnerable parameter was:

filename

The relevant endpoint was:

```
/image ## 5.3 Baseline Request
```

A normal image request was first sent through Burp Repeater:

```
GET /image?filename=5.jpg
```

The server returned a successful HTTP response containing image data.

5.4 Absolute Path Testing

The filename parameter was then changed from the normal image filename to:

```
/etc/passwd
```

This test did not use relative ../ traversal sequences.

5.5 Result

The server returned an HTTP 200 response containing the contents of `/etc/passwd`.

The response included the root account entry:

```
root:x:0:0:root:/root:/bin/bash
```

This demonstrated that accepting absolute filesystem paths can provide a path traversal bypass when validation focuses only on relative traversal sequences.

5.6 Evidence

03-lab-2-initial-page.png - Initial laboratory page 04-lab-2-path-traversal-success.png - Successful absolute-path bypass response
6. Findings Summary Lab Endpoint Parameter Technique
Observed Result Lab 1 /image filename Relative path traversal /
etc/passwd contents returned Lab 2 /image filename Absolute
path bypass /etc/passwd contents returned # 7. Security
Observations

The two exercises demonstrate how applications that construct filesystem paths from user-controlled input can expose local files when path validation is insufficient.

The first laboratory demonstrated direct relative path traversal using `../` sequences.

The second laboratory demonstrated that blocking traversal sequences alone is not sufficient when absolute filesystem paths are still accepted.

The impact of such a vulnerability depends on the permissions of the application process and the files accessible to it. Exposed files may contain system information, configuration data, credentials, application secrets, or other sensitive information.

8. Recommended Mitigations

The following defensive measures can reduce the risk of path traversal vulnerabilities:

Avoid direct use of user-controlled filesystem paths. Where possible, map user-supplied identifiers to predefined server-side filenames. Use an allowlist. Accept only known filenames, identifiers, or permitted resource names rather than arbitrary

filesystem paths. Reject absolute paths. User-controlled input should not be allowed to specify arbitrary absolute filesystem locations. Canonicalize and normalize paths. Resolve the requested path before access and validate the resulting canonical path. Verify directory containment. Ensure the final resolved path remains inside the intended application directory. Apply least-privilege permissions. The web application should have access only to the files and directories required for its operation. Monitor suspicious requests. Requests containing traversal patterns or unexpected filesystem paths should be logged and investigated. Use secure file-handling APIs. Application frameworks and libraries should be configured to safely handle file paths and user input. # 9. Evidence and Documentation

The Day 17 evidence directory contains the following screenshots:

screenshots/ |— 01-lab-1-initial-page.png |— 02-lab-1-path-traversal-success.png |— 03-lab-2-initial-page.png |— 04-lab-2-path-traversal-success.png

The raw task evidence is additionally documented in:

logs/day17-evidence.txt # 10. Limitations Testing was limited to the authorized PortSwigger Web Security Academy laboratories. No production or third-party systems were tested. The demonstrations were performed specifically for educational and security-training purposes. The documented results represent the behavior observed during the laboratory exercises. # 11.
Conclusion

Day 17 provided practical experience identifying and validating path traversal vulnerabilities using Burp Suite.

The first laboratory demonstrated relative path traversal through the filename parameter. The second demonstrated an absolute-path bypass when relative traversal sequences were restricted.

The exercises reinforced the importance of strict input validation, canonical path handling, directory containment checks, allowlisting, and least-privilege filesystem permissions when developing applications that interact with user-controlled file paths.

12. Completion Status

Day 17 - Path Traversal Lab: COMPLETED

Both assigned PortSwigger Web Security Academy laboratories were successfully tested within the authorized training environment, documented with supporting evidence, and prepared for repository submission.